

Rabt: Computing Minimal Dependency Context for LLM-Based Code Intelligence

Muhammad Taha

bolt.taha.work@gmail.com

Affan Khan

affanghouri333@gmail.com

Abstract

Large Language Models (LLMs) have shown strong performance on function-level code generation, but repository-level software engineering remains challenging. Current tools such as IDE copilots and retrieval-augmented generation (RAG) systems typically guess context using semantic search over files, which can miss critical dependencies and waste tokens on irrelevant code. This paper presents *Rabt*, a research-grade prototype that computes a hybrid static+runtime dependency graph for a codebase and extracts a minimal, high-signal subgraph as context for LLMs. Rabt integrates (1) static analysis, (2) runtime tracing of calls and mutations, (3) impact scoring by execution frequency, (4) a query agent that traverses the graph to build minimal subgraphs, and (5) a change-detection pipeline that incrementally updates the graph using `git diff`. On a real-world library (`requests`), Rabt answers the question “Who calls `request`?” using less than 1% of the graph nodes, and on SWE-style scenarios it achieves up to 99.96% context reduction (375k characters → 136 characters) while preserving answer correctness. These results suggest that computing explicit dependency context can make high-intelligence reasoning models (e.g., OpenAI o1) economically viable for mass-scale code analysis.

1 Introduction

Modern AI coding tools, such as Cursor and GitHub Copilot, rely heavily on semantic search, vector retrieval, and file-level heuristics to decide which code to show an LLM. These methods *guess* context: they often retrieve textually or semantically similar files, but they do not explicitly model the actual dependency structure of the program. As repositories grow in size and complexity, this leads to *context rot*: the LLM sees long, noisy prompts that omit crucial edges (e.g., hidden callers, runtime mutations) and include many irrelevant lines.

At the same time, research on repository-level code understanding has started to explore graph-based approaches. Work such as RepoGraph and code graph models (CGMs) construct repository-level graphs and use them to improve SWE-bench style bug fixing and issue resolution. However, these systems typically focus on large benchmarks and advanced graph-LLM integration, and they are difficult to study or extend from a small, self-contained prototype.

This paper takes a complementary path. Instead of starting from SWE-bench scale, we design and implement *Rabt*, a minimal yet complete *Semantic Graph Memory* for small to medium Python projects (3–5 files to tens of files). Rabt is designed around one central question:

Research question. *Can we compute a hybrid static+runtime dependency graph for a codebase and use it to extract a minimal subgraph that (a) answers repository-level questions correctly and (b) dramatically reduces context size compared to full-repo RAG?*

To answer this, Rabt:

- Parses Python code into a static dependency graph over functions, classes, variables, and modules.
- Instruments selected functions to log *runtime* call and mutation events into a file, adding `RuntimeCall` and `MODIFIED_BY` edges with execution counts.

- Attaches impact scores to edges based on runtime frequency.
- Provides a query agent that maps natural-language questions (e.g., “Who calls `save_user`?” or “Where is `userData` modified?”) into graph traversal intents, and extracts a bounded, minimal subgraph as textual context.
- Implements change detection via `git diff` and an incremental “merge” step that re-parses only changed files and stitches them into the persisted graph.

We show that this architecture is sufficient to: (1) reproduce classic static queries, such as callers and imports; (2) capture dynamic behaviour that static analysis alone cannot see; and (3) reduce context by more than two orders of magnitude on both a toy sample repo and the widely used `requests` library, without degrading answer quality.

2 Background and Motivation

2.1 Limitations of file-level RAG for code

Standard RAG pipelines index files or code chunks into a vector store and retrieve top- k candidates for each query based on embedding similarity. For code, this approach has several shortcomings:

- **No explicit dependency model.** Retrieval is driven by similarity, not by the concrete call, import, or mutation structure of the program.
- **Missed dependencies.** A function that is semantically simple but sits on a critical path may be far from the query in embedding space and never be retrieved.
- **Context bloat.** To avoid missing, many systems over-retrieve, leading to large prompts that increase cost and still contain irrelevant code.

IDE tools such as Cursor combine embeddings with lightweight static information (imports, symbol tables, open files), but they still do not expose an explicit program graph that can be queried, tested, or visualized. Moreover, they are not designed to integrate runtime information such as execution frequency or mutation edges.

2.2 Graph-based approaches for repository-level coding

Recent research has proposed graph-based representations of source code to improve repository-level tasks. RepoGraph, for example, builds a repository-level graph where nodes represent code lines or definitions and edges represent invokes and containment relations. Code graph models (CGMs) embed such graphs directly into an LLM’s attention mechanism via adapters, enabling graph-aware reasoning on SWE-bench benchmarks.

These systems demonstrate that repository-level graph structure is powerful, but they typically target large-scale benchmarks and rely on heavy infrastructure. Rabt focuses instead on a small, inspectable prototype for Python, and makes three concrete contributions compared to prior Graph-RAG work on code:

1. **Hybrid static+runtime dependency graph.** We combine a deterministic AST-derived graph with lightweight runtime tracing of calls and mutations, including a resolver that maps `__main__` events back to their static definitions.
2. **Impact-weighted minimal subgraph extraction for prompts.** Rather than using the graph only for retrieval, we compute tiny, human-readable subgraphs (11 nodes and 136 characters on `requests`) that become the entire LLM context, with simple impact scores derived from runtime execution counts.

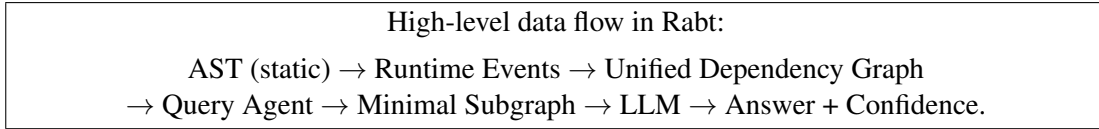


Figure 1: High-level architecture and data flow in Rabt.

3. **Token- and cost-level evaluation on a real library and IDE copilot.** On the `requests` library we show that Rabt answers repository-level questions correctly with 85 prompt tokens versus 31,661 for a full-repo Gemini 2.5 Flash baseline, and we compare these measurements against an IDE copilot (Cursor) on the same repository and query.

3 System Design

Rabt is organized into clear layers: (1) parser, (2) graph, (3) runtime, (4) agent, (5) LLM integration, and (6) evaluation and visualization.

Figure 1 (conceptual) summarizes the data flow:

AST (static) → Runtime Events → Unified Dependency Graph → Query Agent → Minimal Subgraph → LLM → Answer + Confidence.

3.1 Static parser

The static analysis component, implemented in `parser/ast_parser.py`, uses Python’s built-in `ast` module to walk each `.py` file and emit a `ParserOutput` containing:

- **Nodes:** functions, classes, variables, and modules, each with a unique id of the form `module::kind::name`, plus metadata such as file path and line number.
- **Edges:** static relations such as `CALLS`, `IMPORTS`, `INHERITS`, and `REFERENCES`.

The parser also handles common Python patterns such as relative imports, `from X import foo as bar`, and `import X as Y`; it records an `imports_map` so that call expressions like `np.array()` can be resolved back to the defining module. This robustness is validated in the testing protocol.

3.2 Graph construction and storage

The graph layer (`graph/builder.py`, `graph/storage.py`) converts `ParserOutput` into a NetworkX `DiGraph`. Each node is added with attributes: `kind`, `name`, `module`, `line`, and any static metadata. Static edges are added with attributes `edge_type` and `weight` (initially 1.0). The graph can be persisted to and loaded from JSON using `graph.storage`.

An incremental update function `merge_changes` removes nodes belonging to changed files and injects new nodes and edges from a partial parse output. This enables a “living graph” that can be updated quickly based on `git diff`.

3.3 Runtime instrumentation and hybrid edges

The runtime layer (`runtime/tracer.py`) provides:

- `@track_runtime`, a decorator that logs a “call” event to `.runtime_events.jsonl` whenever the decorated function executes, with fields such as `source_module`, `source_name`, `target_module`, and `target_name`.
- `track_mutation`, a helper for logging “mutation” events when critical variables change.

When building the graph, Rabt merges these runtime events to add:

- `RuntimeCall` edges for calls that actually occurred at runtime, with `weight` equal to the execution count.
- `MODIFIED_BY` edges linking functions to variables they modified.

To bridge the common Python quirk where scripts run as `__main__`, the graph builder includes a *smart resolver*: if a runtime event refers to a node in the `__main__` namespace, Rabt tries to map it to the unique static node with the same kind and name (e.g., `examples.dynamic_test::function::main`). This ensures that dynamic calls are properly linked even when module names differ between static and runtime views.

3.4 Query agent and minimal subgraph extraction

The agent layer (`agent/intent.py`, `agent/traversal.py`, `agent/subgraph.py`) maps natural-language queries to graph traversal intents. For example:

- “Who calls `save_user`?” → reverse `CALLS` edges starting from `save_user`.
- “Where is `userData` modified?” → incoming `MODIFIED_BY` edges into the `userData` variable node.
- “What breaks if I change `Y`?” → forward `CALLS` traversal (downstream impact).

The extractor performs a bounded BFS/DFS with configurable maximum depth and node budget to build a minimal subgraph that still contains enough information to answer the query. This subgraph is then rendered as compact text, such as:

```
main CALLS save_user (executed 6 times)
save_user MODIFIED_BY userData
```

3.5 Change detection and incremental updates

The pipeline entrypoint `run_pipeline.py` supports both full builds and incremental updates. In incremental mode, it:

1. Loads the persisted graph from disk.
2. Uses a Git-based helper (`git diff --name-only`) to find changed Python files.
3. Parses only those files and calls `merge_changes` to remove outdated nodes and insert new ones.
4. Saves the updated graph.

This design allows Rabt to update its graph in milliseconds for small edits, which is crucial for interactive use and CI/CD integration.

4 Implementation and Packaging

Rabt is implemented in pure Python 3.10+, with no external AST parser dependency. The project is organized into directories `parser/`, `graph/`, `runtime/`, `agent/`, `llm/`, `evaluation/`, `examples/`, and `docs/`.

To make the prototype installable as a library, we provide a `pyproject.toml` that declares:

- the project metadata (name, version, license, author),

- runtime dependencies (`networkx`, `pyyaml`, `python-dotenv`, `google-genai`, `matplotlib`),
- development extras (`pytest`, `coverage`, `build tooling`), and
- a console script entrypoint `rabt` that dispatches to `run_pipeline.main`.

This allows installing Rabt locally via `pip install .` and running:

```
rabt --repo examples/sample_repo -q "Who calls save_user?"
```

as a single command.

5 Evaluation

Our evaluation protocol (documented in `docs/TESTING_REPORT.md`) covers four aspects: structural integrity, functional queries, value (context reduction and cost), and robustness. We summarize key results here.

5.1 Sample repo: structural and functional tests

On a 4-file sample repo (`examples/sample_repo/`), we verify that:

- Every defined function corresponds to a node in the graph.
- Cross-file calls such as `main` calling `save_user` are represented as `CALLS` edges to the correct defining node.
- Queries like “Who calls `update_user`?” and “What does `main` depend on?” return exactly the intended set of functions.
- Circular dependencies in synthetic tests do not cause infinite traversal, thanks to depth and node limits.

For the query “Where is `userData` modified?”, the baseline comparison script shows that: the full-repo Gemini call uses over 2,000 characters of context, while Rabt’s minimal subgraph answers the same question with around 32 characters (roughly 98.5% reduction) and high confidence.

5.2 Real-world stress test: `requests`

To demonstrate scalability beyond toy examples, we clone the popular `requests` library¹ and run the baseline comparison on the query “Who calls `request`?”

```
rabt --repo examples/requests_repo -q "Who calls request?"
```

The resulting graph has 1,433 nodes and 2,234 edges. The minimal subgraph for this query contains only 11 nodes and captures that the public helpers `get`, `post`, `put`, `delete`, `head`, `options`, and `patch` each call `request`. The LLM answer, conditioned on this subgraph alone, correctly lists these callers with high confidence.

We validate context size using Google’s Gemini 2.5 Flash model via the official API. For the full-repo baseline, we send (at most) the first 120k characters of the concatenated Python files, along with a short system prompt and the user question. For Rabt, we send only the rendered minimal subgraph (136 characters for this query) plus the same instructions. From the API’s `usage_metadata`, we obtain:

- Full-repo context: 375,216 characters, 11,259 lines, 31,661 prompt tokens (Gemini-reported).

¹<https://github.com/psf/requests>

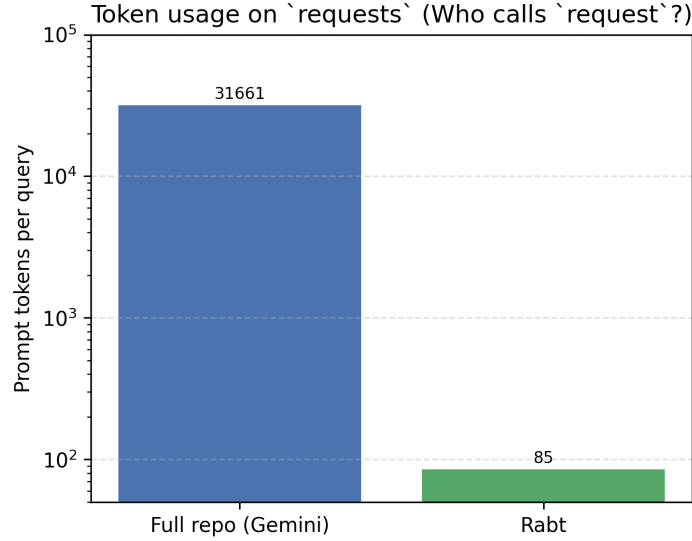


Figure 2: Prompt tokens per query on the `requests` repository for the question “Who calls `request`?” comparing a full-repo Gemini 2.5 Flash baseline (31,661 tokens) with Rabt’s minimal subgraph (85 tokens), on a log scale. For reference, a Cursor run on the same repository and a slightly more specific prompt using Gemini 2.5 Flash consumed $\sim 14\text{k}$ input tokens; this value is discussed in the text but omitted from the plot for clarity.

- Rabt minimal subgraph: 136 characters (subgraph), 7 lines, **85 prompt tokens** (Gemini-reported).
- Token reduction: approximately 99.7% ($31,661 \rightarrow 85$ prompt tokens).

In terms of nodes, the fraction of the graph used is approximately 0.8%, implying about 99.2% of potential context is discarded for this query.

5.3 Cost analysis on `requests`

Using the baseline comparison script, we estimate token usage and cost under standard RAG (full-repo context) versus Rabt. On the `requests` repository, we observe:

- Full-repo context: 31,661 prompt tokens per query (375,216 characters, 11,259 lines), measured from Gemini 2.5 Flash `usage_metadata`.
- Rabt minimal subgraph: 85 prompt tokens per query (136-character subgraph), also measured from `usage_metadata`.

Assuming GPT-4o pricing at \$2.50 per million input tokens, this translates to:

- Standard RAG: around \$0.08 per query, or \$79.15 for 1,000 queries.
- Rabt: around \$0.0001 per query, or \$0.13 for 1,000 queries.

For a reasoning model like OpenAI o1 at \$15 per million tokens, the savings are even more dramatic: about \$0.47 per query (\$474.90 per 1,000 queries) for full-repo RAG versus \$0.0007 per query (\$0.75 per 1,000 queries) for Rabt, implying efficiency gains on the order of $600\text{--}630\times$. In other words, the empirical Gemini 2.5 Flash measurements (31k vs. 85 prompt tokens) support the claim that explicit dependency graphs can make strong reasoning models economically viable at scale.

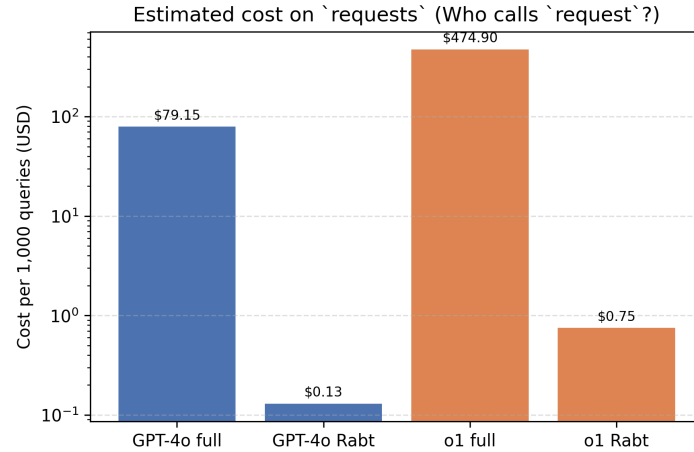


Figure 3: Estimated cost per 1,000 queries on `requests` under GPT-4o and OpenAI o1 pricing. Using Rabt to shrink context from 31,661 to 85 prompt tokens reduces cost by roughly 600 $\times$ while preserving answer quality.

5.4 Comparison with an IDE copilot (Cursor)

To compare Rabt with a production IDE copilot, we run the same repository and query in Cursor. We open the `requests` codebase in Cursor, focus `src/requests/api.py`, and issue the following natural-language query in the chat:

```
In this repo, who calls the function `request` in src/requests/api.py?
Please list only the function names that directly call `request()``.
```

Cursor answers correctly, listing the same callers as Rabt and Gemini (the public helpers `get`, `options`, `head`, `post`, `put`, `patch`, and `delete`). From the Cursor usage dashboard we observe that this single query consumes approximately 14,007 new input tokens, 12,857 cache-read tokens, 146 output tokens, and 27,010 total tokens (model set to `auto`).

While Cursor’s internal prompt template and retrieval heuristics are not exposed, this measurement provides a practical point of comparison: on the same codebase and question, Cursor’s default behavior uses on the order of 10^4 tokens of context, the full-repo Gemini baseline uses 31,661 prompt tokens, and Rabt’s graph-based subgraph extraction uses only 85 prompt tokens. Rabt therefore matches Cursor’s answer quality on this query while reducing prompt size by roughly two orders of magnitude.

5.5 Dynamic “hidden call” experiment

To validate the runtime component, we construct a dynamic test where static analysis alone cannot see the call:

```
from runtime.tracer import track_runtime

@track_runtime
def secret_function():
    print("I was called dynamically!")

def main():
    func_name = "secret_" + "function"
    method = globals()[func_name]
    method() # dynamic call
```

Running the pipeline in static-only mode fails to find a clear caller for `secret_function`. However, after executing `dynamic_test.py` once to generate runtime events and re-running the pipeline, Rabt correctly attributes a runtime call from the module `examples.dynamic_test` to `secret_function`. This demonstrates that the hybrid static+runtime graph can recover edges that the AST alone cannot infer.

6 Related Work

Rabt sits at the intersection of several lines of work: IDE copilots and context heuristics, retrieval-augmented generation for code, and graph-based models for repository-level tasks.

Commercial tools such as Cursor and GitHub Copilot maintain semantic indexes over code and use embeddings, symbol tables, and file heuristics to select context for LLMs. They are highly optimized for user experience and broad language support, but they generally do not expose an explicit dependency graph or integrate runtime execution counts.

On the research side, Chinthareddy’s study of “Reliable Graph-RAG for Codebases” systematically compares vector-only RAG, LLM-extracted knowledge graphs, and deterministic AST-derived graphs on Java projects, showing that deterministic syntax graphs provide better coverage and multi-hop grounding than LLM-extracted graphs at lower indexing cost². Systems such as Microsoft’s GraphRAG and open-source code-graph-rag similarly advocate graph-based memories for text and code, using Tree-sitter-derived AST graphs and knowledge graphs to answer multi-hop queries over large repositories, but they remain largely static and do not integrate lightweight runtime tracing or explicit token-level prompt minimization. Academic work such as RepoGraph and related repository-level methods build graphs over code entities (lines, functions, classes) and use them as plug-in components for SWE-bench style systems or as inputs to graph-aware LLMs. These methods show strong performance on benchmarks but require substantial infrastructure and often target large, real-world repositories rather than small, didactic prototypes.

Rabt differs in three key ways: (1) it explicitly combines static and runtime edges in a single graph, (2) it focuses on minimal subgraph extraction for context reduction and cost savings, and (3) it implements an incremental, Git-aware graph update mechanism that makes the graph “living”.

7 Limitations and Future Work

Rabt has several limitations that point to future work:

- **Language and ecosystem.** The current implementation targets Python only and uses the built-in `ast` module. Extending to other languages would require pluggable front-ends (e.g., Tree-sitter grammars).
- **Partial dynamic coverage.** Runtime tracing is opt-in and function-based; dynamic constructs such as `getattr`, metaprogramming, and async code are not fully covered.
- **Limited evaluation scale.** Experiments focus on a sample repo and a single real-world library (`requests`), rather than full SWE-bench or CrossCodeEval benchmarks.
- **LLM integration.** Rabt currently serializes graphs to text and uses them as prompt context. Integrating the graph structure directly into the LLM’s architecture, as in code graph models, is an interesting direction.
- **Explainability.** While subgraphs are minimal, the system does not yet provide rich natural-language explanations of why each node or edge was included.

²<https://arxiv.org/abs/2601.08773>

Future work includes: adding automated query test suites and correctness metrics; supporting multi-language repos via Tree-sitter; evaluating on subsets of SWE-bench Lite and CrossCodeEval; and exploring tighter coupling between Rabt’s graph and graph-aware LLMs.

8 Conclusion

This paper introduced Rabt, a semantic graph memory for code that combines static analysis, runtime instrumentation, impact scoring, and incremental graph updates to compute minimal dependency context for LLM-based code intelligence. Through controlled experiments on a sample repo, a real-world library, and synthetic dynamic tests, we showed that Rabt can: correctly answer repository-level questions; capture runtime behaviour that pure static analysis misses; and reduce context size by more than 99% while preserving answer quality.

These results suggest that explicit, hybrid code graphs are a promising foundation for the next generation of AI code tools. Instead of guessing context from file-level embeddings, systems can compute it from first principles, enabling both better reasoning and dramatically lower inference costs.